



도커를 사용하는 이유

- 도커를 사용하면 해결

Python 버전 및 라이브러리 격리
개인/회사 PC 간 동일한 환경 구성
환경 꼬임 시 컨테이너 삭제 후 즉시 복구
검증된 이미지로 카메라/GUI 오류 해결
하나의 PC에서 다양한 Ubuntu/ROS 버전 운영

Windows 10 64-bit
가능하면 22H2 권장
WSL2 사용 가능 상태

구분	설명
이미지 (Image)	실행에 필요한 모든 파일이 담긴 읽기 전용 압축 파일
컨테이너 (Container)	이미지를 격리된 환경에서 실제로 실행시킨 상태
볼륨 (Volume)	컨테이너 삭제 후에도 데이터가 유지되는 외부 저장소

1

3. 실행 (Run)	도커 컨테이너	도커 안에서 코드를 빌드하고 로봇(Node)을 깨워 MuJoCo/Gazebo 등 가상 세계에서 실제로 작동.
-------------	---------	--

* 명령어 입력 전에 항상 Docker를 켜고 진행할 것.

도커 기본 명령어

```
# 1. 이미지 관리
docker pull [이미지명]      # 도커 허브에서 이미지 다운로드 ex. docker pull osrf/ros:hum
ble
docker images              # 내 PC에 있는 이미지 목록 확인
docker inspect [이름/ID]   # 컨테이너/이미지의 상세 정보(IP, 설정 등) 확인

# 2. 실행 및 상태 확인
docker run [이미지명]      # 이미지를 컨테이너로 생성 + 실행
docker ps                  # 현재 실행 중인 컨테이너 목록
docker ps -a               # 중지된 컨테이너를 포함한 전체 목록 확인
docker logs [컨테이너ID]   # 컨테이너 내부 출력 로그 확인
docker logs -f [컨테이너ID] # 특정 컨테이너의 로그를 실시간으로 계속 출력 (-f: 실시간 추적)

# 3. 제어 및 삭제
docker stop [컨테이너ID]   # 실행 중인 컨테이너 중지
docker start [컨테이너ID]  # 중지된 컨테이너 다시 시작
docker rm [컨테이너ID]     # 컨테이너 삭제 (중지 상태여야 함)

# 4. 내부 명령 실행
docker exec -it [컨테이너ID] bash # [작동 중인] 컨테이너 내부로 접속하여 터미널(bash) 실행
```

* 대괄호는 제외하고 작성할 것

도커 핵심 명령어 요약

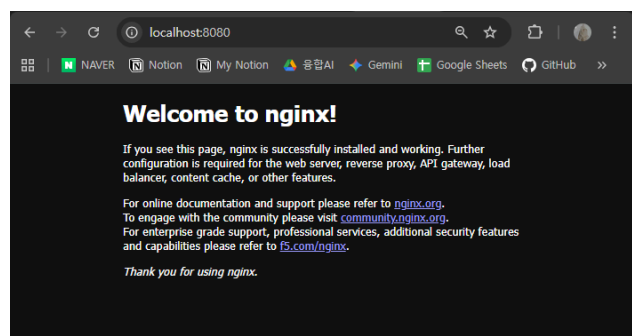
1) 웹 서버 실행 (포트 연결)

```
docker run -d -p 8080:80 --name web nginx
```

- 해석: 내 컴퓨터 8080포트와 컨테이너 80포트를 연결하여 Nginx 서버를 백그라운드로 실행함.
- 포트포워딩(-p) 안 쓰면 외부접속이 아예 불가능

👉 접속: <http://localhost:8080>

- local host 접속 예시 이미지



2) OS 컨테이너 백그라운드 유지

```
docker run -it -d --name ubuntu ubuntu:22.04 bash
```

- **해석:** 우분투를 백그라운드에서 꺼지지 않게 '대기 상태'로 켜둠.
- **핵심:** `-d` 를 써서 뒷마당(백그라운드)에서 서버가 계속 돌게 함. `-it` 덕분에 "사용자가 언제든지 들어올 수 있게 터미널 준비하고 대기해!"라는 상태가 됨.

👉 뒤에 `bash` 붙이면 바로 터미널창 열리게 함.

3) 실행 중인 컨테이너 접속 (들어가기)

```
docker exec -it ubuntu bash # execute : 실행하라는 약어.
```

- **해석:** 이미 켜져 있는(run 상태) 컨테이너 안으로 직접 들어감. (예시는 ubuntu라는 컨테이너)
- **비유:** `run` 은 새 집 짓기, `exec` 은 지어진 집 방문하기임.

4) 내 파일 공유 (볼륨 마운트)

```
docker run -it --name my_ros -v C:/work/docker_ws:/ros_ws ubuntu:22.04 bash
```

- **해석:** [내 컴퓨터 폴더]:[컨테이너 폴더] 를 하나로 연결.
- **장점:** 컨테이너를 지워도 내 컴퓨터 하드에 작업물이 남음. (ROS 개발 필수)

docker-compose

1) docker-compose.yaml (설계도)

- 긴 명령어를 칠 필요 없이 `docker-compose.yaml` 파일 하나로 여러 컨테이너를 일괄 관리함.

```
version: "3.8"

services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

- **내용:** 이미지 이름, 포트 번호 등 실행에 필요한 모든 정보를 적어둔 파일임.
- **장점:** 긴 명령어를 매번 칠 필요 없이 파일 하나로 관리함.

👉 `docker-compose.yaml` 파일은 띄어쓰기에 굉장히 예민함. (클론, 대시 뒤에는 띄어쓰기로 구분해주기.)

2) 주요 명령어

```
# 1. 일괄 관리
docker-compose up           # 설정파일(yaml)대로 모든 서비스 생성 및 실행
docker-compose up -d       # (서버 켜기) 백그라운드 모드로 실행 (터미널 계속 사용 가능)
docker-compose down        # (서버 닫기) 모든 서비스 중지 및 생성된 컨테이너 삭제

# 2. 빌드 및 확인
```

```
docker-compose build      # Dockerfile 수정 시 이미지를 새로 다시 빌드
docker-compose ps         # Compose로 관리되는 서비스들의 상태 확인
docker-compose logs -f    # 모든 서비스의 로그를 실시간으로 모니터링 (-f)
```

3. 특정 서비스 제어

```
docker-compose exec [서비스명] [명령어] # 특정 컨테이너 안에 들어가 명령 실행
```

✨ 필요한 파일들 다 만들었다면, 아래 두 줄만 해서 진입. (build + 컨테이너 생성)

```
docker-compose build      # Dockerfile에서 지정된 이미지들 가져오고
```

```
docker-compose up -d      # 컨테이너 생성
```

✨ docker와 내 pc를 볼륨으로 연결시키고, 이것저것 작업했다가 실수로 컨테이너를 삭제했다면, 다시 build하고 up하면 다시 docker에 내 pc와 똑같이 연동됨.

- "작업실(도커)은 새로 지어도 설계도와 결과물(내 PC)은 그대로"인 구조

▼ ROS2

ros2 docker-compose.yaml 파일 형식

```
services:
  ros2:
    image: osrf/ros:humble-desktop      # osrf에서 ros humble과 관련 이미지들 가져옴.
    container_name: ros2_container      # 여기에 원하는 이름 작성
    stdin_open: true                    # 터미널 입력 열어두기
    tty: true                           # 컨테이너가 안 꺼지게 대기시키기

    volumes:
      - ./workspace:/ros_ws             # [내 컴퓨터 폴더]:[컨테이너 폴더] 연결

    working_dir: /ros_ws                 # 접속 시 시작할 폴더 지정

    command: bash                       # 시작하자마자 터미널(bash) 실행
```

cf) osrf : Open Source Robotics Foundation : ROS를 만드는 공식 단체 계정

💡 꼭 docker-compose.yaml 파일이 있는 곳에서 실행시켜줘야 함.

```
docker-compose up -d      # 모든 컨테이너를 한 번에 생성하고 서버실행 (백그라운드 모드)
docker-compose exec ros2 bash # 실행 중인 'ros2' 서비스 내부로 들어가서 터미널(bash)을 실행함
```

* 컨테이너ID를 입력하는게 아니라, Services 이름을 입력해야 함.

docker-compose 이후, ROS2 패키지 생성 및 빌드 과정 요약

1) ROS2 환경 확인

```
source /opt/ros/humble/setup.bash # 터미널 열 때마다 해야하기 때문에 ~/.bashrc에 넣어줘야 함.  
ros2 --version
```

2) 패키지 생성

```
ros2 pkg create --build-type ament_python my_pkg
```

- 해석: Python 기반의 ROS2 패키지(`my_pkg`)를 자동으로 만들어줌.
- 결과: `setup.py`, `package.xml` 등 필수 파일들이 담긴 폴더가 생성됨.

3) 빌드 실행

```
colcon build
```

- 명령: `/workspace` 경로에서 실행함.
- 해석: 내가 만든 소스코드(`src`)를 컴퓨터가 실행할 수 있는 형태로 변환(컴파일/설치)함.
- 결과: `build`, `install`, `log` 폴더가 자동으로 생김.
 - `install/`: 실제로 실행될 파일들이 모이는 곳임.
 - `build/`: 빌드 과정 중 발생하는 임시 파일 저장소임.

💡 빌드 후 필수 단계 (중요)

빌드가 끝났다고 바로 실행할 수 있는 게 아님. 내가 만든 패키지를 인식시켜줘야 함.

```
# /workspace 폴더에서 실행함  
source install/setup.bash
```

- 해석: "방금 빌드해서 `install` 폴더에 새로 생긴 패키지들을 현재 터미널에 등록해라"라는 뜻임.
- 순서: `colcon build` → `source install/setup.bash` → `ros2 run ...` (이게 한 세트임)

👉 주의 : `src/my_pkg` 여기서 노드를 만들어야 하는데, `install/my_pkg`에서 할 수도 있음.

4) talker / listener 예제 노드 (ROS 기본 제공 - 확인용)

```
## 1번째 터미널창  
ros2 run demo_nodes_cpp talker  
  
## 2번째 터미널창  
ros2 run demo_nodes_cpp listener  
  
## 3번째 터미널창  
ros2 topic echo /chatter # echo = Linux에서 토픽 세부 내용들을 터미널로 출력하는 명령어
```

결과 이미지 (1. talker / 2. listener / 3. echo chatter)

```

root@7164e11bb21f: /workspace
INFO [1777533181.830067931] [talker] Publishing: 'Hello World: 127'
INFO [1777533182.430463303] [talker] Publishing: 'Hello World: 128'
INFO [1777533183.538976380] [talker] Publishing: 'Hello World: 129'
INFO [1777533184.838954638] [talker] Publishing: 'Hello World: 130'
INFO [1777533185.759130189] [talker] Publishing: 'Hello World: 131'
INFO [1777533186.830068189] [talker] Publishing: 'Hello World: 132'
INFO [1777533189.422224944] [talker] Publishing: 'Hello World: 133'
INFO [1777533189.532176734] [talker] Publishing: 'Hello World: 134'
INFO [1777533189.632176704] [talker] Publishing: 'Hello World: 135'
INFO [1777533200.720103544] [talker] Publishing: 'Hello World: 136'
INFO [1777533200.832142634] [talker] Publishing: 'Hello World: 137'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 138'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 139'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 140'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 141'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 142'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 143'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 144'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 145'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 146'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 147'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 148'
INFO [1777533200.933064941] [talker] Publishing: 'Hello World: 149'
INFO [1777533180.431040231] [listener] heard: 'Hello World: 125'
INFO [1777533181.430381741] [listener] heard: 'Hello World: 126'
INFO [1777533182.429518016] [listener] heard: 'Hello World: 127'
INFO [1777533183.53454688] [listener] heard: 'Hello World: 128'
INFO [1777533184.53454688] [listener] heard: 'Hello World: 129'
INFO [1777533185.73057228] [listener] heard: 'Hello World: 130'
INFO [1777533186.83057228] [listener] heard: 'Hello World: 131'
INFO [1777533189.432797224] [listener] heard: 'Hello World: 132'
INFO [1777533189.532885154] [listener] heard: 'Hello World: 133'
INFO [1777533189.632885154] [listener] heard: 'Hello World: 134'
INFO [1777533200.732800624] [listener] heard: 'Hello World: 135'
INFO [1777533200.832800624] [listener] heard: 'Hello World: 136'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 137'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 138'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 139'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 140'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 141'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 142'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 143'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 144'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 145'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 146'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 147'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 148'
INFO [1777533200.932800624] [listener] heard: 'Hello World: 149'

```

5) 실제 노드 생성

파일 경로

```

workspace/
├─ build/
├─ install/
├─ log/
└─ src/
    └─ my_pkg/
        ├── package.xml
        ├── setup.py
        ├── setup.cf
        └─ my_pkg/
            ├── __init__.py
            └─ my_node.py

```

빌드 결과물이 생성되는 곳 (건드리지 않음)

실행 파일이 설치되는 곳 (건드리지 않음)

💡 우리가 작업하는 소스 폴더

빌드 및 엔트리포인트 설정 파일

📍 실제 파이썬 코드가 담기는 곳

★ 여기에 파일을 생성

my_node.py

```

import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node')
        self.get_logger().info("Hello ROS2")

def main():
    rclpy.init()
    node = MyNode()
    rclpy.spin(node)
    rclpy.shutdown()

```

setup.py

```

from setuptools import find_packages, setup

package_name = 'my_pkg'

setup(
    name=package_name,

```

```

version='0.0.0',
packages=find_packages(exclude=['test']),
data_files=[
    ('share/ament_index/resource_index/packages',
     ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
],
install_requires=['setuptools'],
zip_safe=True,
maintainer='root',
maintainer_email='root@todo.todo',
description='TODO: Package description',
license='TODO: License declaration',
extras_require={
    'test': [
        'pytest',
    ],
},
entry_points={
    'console_scripts': [
        'my_node = my_pkg.my_node:main',      # [추가] - 새로운 노드 추가
    ],
},
)

```

빌드 및 실행

```

cd /workspace/
colcon build
source install/setup.bash      # 업데이트한 걸 source로 알려줘야 함.

ros2 run my_pkg my_node      # 노드 실행

```

▼ docker-compose 응용 (MySQL / ROS2 / MuJoCo)

AI 프롬프트

```

" 너는 20년차 로봇 시스템 전문 아키텍처야. "
" docker-compose 기반으로 MySQL 8.0 / ROS2 Humble / MuJoCo 3.6.0 서버를 구성해줘. "
" 초보자도 이해하기 쉽도록 주석문을 사용하여 구성 내역에 대한 상세한 설명을 같이 작성해줘. "

```

생성된 docker-compose.yaml

```

version: '3.8'

services:
  # 1. 데이터베이스 서버: 로봇의 기록을 담당합니다.
  db:
    image: mysql:8.0
    container_name: robot_database
    restart: always
    environment:
      MYSQL_DATABASE: robot_logs

```

```

    MYSQL_ROOT_PASSWORD: password123 # 실제 사용 시 보안을 위해 변경하세요.
ports:
  - "3306:3306"
volumes:
  - ./mysql_data:/var/lib/mysql # 컨테이너가 꺼져도 데이터가 보존되도록 설정

# 2. 로봇 두뇌 (ROS2 Humble): 통신과 제어를 담당합니다.
ros2-humble:
  image: osrf/ros:humble-desktop
  container_name: ros2_master
  stdin_open: true
  tty: true
  network_mode: "host" # ROS2의 원활한 멀티캐스트 통신을 위해 호스트 네트워크 권장
  volumes:
    - ./src:/root/ros2_ws/src # 내 소스코드를 컨테이너와 동기화
  environment:
    - DISPLAY=$DISPLAY # GUI 도구(Rviz 등) 사용을 위한 설정
  depends_on:
    - db # DB가 먼저 켜진 후 실행되도록 설정

# 3. 물리 엔진 (MuJoCo 3.6.0): 가상 세계를 담당합니다.
mujoco:
  build:
    context: .
    dockerfile: Dockerfile.mujoco
  container_name: mujoco_sim
  runtime: nvidia # GPU 가속이 필요한 경우 (NVIDIA Docker 필요)
  environment:
    - DISPLAY=$DISPLAY
  volumes:
    - ./models:/root/.mujoco/models # 로봇 모델 파일(.xml) 경로
  depends_on:
    - ros2-humble

```

▼ OpenCV

OpenCV를 배우는 이유

ROS2 센서 데이터 받은 후,

 카메라 센서
 ↓
 → ROS2 Image Topic
 → OpenCV 변환
 → 영상 전처리

우리가 해야하는 것들

이미지 읽기
 영상 프레임 처리
 색상 공간 변환
 윤곽선 검출
 마커 인식

→ 객체/마커/라인 인식
→ 로봇 제어 판단

카메라 캘리브레이션
객체 추적

🌳 미리보는 최종 환경 파일 tree

```
Windows 10
├─ Docker Desktop
│   └─ WSL2 Backend
├─ Google Antigravity
│   └─ 프로젝트 폴더 열기 / 코드 작성 / 명령 실행 보조
└─ opencv-docker-lab/
    ├─ docker-compose.yml          # 컨테이너 실행 설정
    ├─ Dockerfile                  # Python/OpenCV가 설치된 이미지를 만드는 설계도
    ├─ requirements.txt            # Python 패키지 실행 목록
    ├─ app/                        # 실습 Python 코드 저장
    │   ├─ 01_check_opencv.py
    │   ├─ 02_image_read_show.py
    │   └─ 03_camera_like_video.py
    ├─ data/                      # 실습 이미지, 영상 저장
    │   ├─ images/
    │   └─ videos/
    ├─ outputs/                   # 처리 결과 이미지/영상 저장
    └─ notebooks/                 # Jupyter Notebook 실습 저장
```

opencv-docker-lab/

data/

app
data
notebooks
outputs
docker-compose.yml
Dockerfile
requirements.txt

images
videos

*** 무조건 이 이름 그대로 쓸 것**
(나는 work 폴더 밑에 만들)

📄 requirement.txt 양식

```
numpy==2.2.6          # numpy는 최신 버전이 마냥 좋진 않음.
opencv-python-headless==4.12.0.88
matplotlib==3.10.5
pillow==11.3.0
jupyterlab==4.4.6

# 주의💡 = 2개 쓸 것
```

이렇게 이미지를 표시하면 안 됨. ❌

```
cv2.imshow("image",img)
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

! Docker는 GUI 출력 드라이버가 없어서 Window 처럼 하면 ERROR 뜨면서 Shutdown 됨.

- 대신 이렇게 출력

```
cv2.imwrite("/workspace/outputs/result.jpg",img)
```

Docker의 result.jpg 주소

or Jupyter Notebook에서 matplotlib 로 출력

```
import matplotlib.pyplot as plt  
plt.imshow(img)  
plt.show()
```

cv2.imshow가 안 되지 plt.imshow는 됨.

Dockerfile 양식

```
FROM python:3.11-slim
```

```
# 컨테이너 내부 작업 폴더
```

```
WORKDIR /workspace
```

```
# Python 실행 시 로그가 바로 출력되도록 설정
```

```
ENV PYTHONUNBUFFERED=1
```

```
# pip 최신화 및 기본 패키지 설치
```

```
RUN python -m pip install --upgrade pip
```

```
# requirements.txt를 먼저 복사
```

```
# 이렇게 하면 requirements.txt가 바뀌지 않았을 때 Docker 캐시를 활용할 수 있음
```

```
COPY requirements.txt /workspace/requirements.txt
```

```
# Python 패키지 설치
```

```
RUN pip install --no-cache-dir -r /workspace/requirements.txt
```

```
# 기본 실행 위치
```

```
CMD ["bash"]
```

docker-compose.yml 양식

💡 yml파일은 들여쓰기를 TAB 말고 띄어쓰기 2번으로 구분.

```
services:
```

```
  opencv-lab:
```

```
    build:
```

```
      context: .
```

docker-compose.yml 파일이 있는 현재 폴더를 기준으로 빌드

```
      dockerfile: Dockerfile
```

dockerfile을 Dockerfile로

```
    image: opencv-lab:1.0
```

생성될 이미지 이름과 버전(태그)

```
    container_name: opencv-lab-container
```

```
    working_dir: /workspace
```

컨테이너 접속 시 기본 시작 폴더

[중요🌟] 호스트(윈도우)와 컨테이너(리눅스) 폴더 연결

```

volumes:
  - ./app:/workspace/app          # 파이썬 코드 저장 폴더
  - ./data:/workspace/data        # 입력 이미지/영상 데이터 폴더
  - ./outputs:/workspace/outputs  # cv2.imwrite 결과물 저장 폴더
  - ./notebooks:/workspace/notebooks # 주피터 노트북 파일 폴더

ports:
  - "8888:8888" # 주피터 노트북 접속을 위한 포트 개방

stdin_open: true # 컨테이너 표준 입력 유지
tty: true # 터미널(bash)을 쓰기 위한 가상 단말기 활성화

command: bash # 실행 시 터미널 모드로 대기

```

빌드 및 컨테이너 생성 / 진입

```

docker-compose build # Dockerfile에서 지정된 이미지들 가져오고

docker-compose up -d # 컨테이너 생성

docker-compose exec opencv-lab bash # 컨테이너 진입 (서비스명 : opencv-lab)

```

AI 프롬프트 예시

- 나는 Antigravity 사용

현재 프로젝트 폴더에 **docker-compose** 기반 OpenCV Python 실습 환경을 만들고 싶습니다.
다음 조건에 맞게 Dockerfile, docker-compose.yml, requirements.txt를 검토해 주세요.

조건:

- Windows 10 + Docker Desktop 환경
- Python 3.11 기반
- opencv-python-headless 사용
- app, data, outputs, notebooks 폴더를 각각 /workspace 하위로 볼륨 연결
- JupyterLab 포트는 8888 사용
- 컨테이너 기본 작업 폴더는 /workspace
- 초보자가 이해하기 쉽게 주석을 추가

DeepL 번역기로 영어로 바꿔 입력하는게 좋음. (데이터 적게 먹음.)

+ 한국어는 존댓말로 하는게 좋음. (그렇게 학습함.)

Tip 💡 Antigravity의 오른쪽 상단에 ... 누르고 Customization 에 Global로 "모든 답변은 한국어로 해줘."로 설정

▼ 예시 1) Python Code

```

# app/01_check_opencv.py

"""
첫 번째 OpenCV 실행 테스트 파일

목표:
1. Python 실행 확인
2. OpenCV(cv2) 설치 확인
3. NumPy 설치 확인

```

4. 테스트 이미지 생성
5. OpenCV로 이미지 저장/읽기/변환
6. 결과를 outputs 폴더에 저장

주의:

- Docker 컨테이너 내부 기준 경로는 /workspace 입니다.
- app, data, outputs 폴더는 docker-compose.yml에서 볼륨으로 연결되어 있습니다.

```
import os
import cv2
import numpy as np    # opencv도 numpy를 기본으로 사용.(행렬 사용 위함)
```

```
def ensure_dir(path: str) -> None:
    """
    폴더가 없으면 생성하는 함수입니다.
    Docker 볼륨 연결 상태에서도 안전하게 사용할 수 있습니다.
    """
    os.makedirs(path, exist_ok=True)

def main():
    print("=== OpenCV Docker Lab 실행 테스트 ===")

    # 1. 버전 확인
    print(f"OpenCV version: {cv2.__version__}")
    print(f"NumPy version: {np.__version__}")

    # 2. 컨테이너 내부 기준 경로 설정
    image_dir = "/workspace/data/images"
    output_dir = "/workspace/outputs"

    ensure_dir(image_dir)
    ensure_dir(output_dir)

    # 3. 테스트 이미지 생성
    # 300x500 크기의 검은색 이미지 생성
    height = 300
    width = 500
    image = np.zeros((height, width, 3), dtype=np.uint8)

    # 4. 이미지 위에 도형과 글자 그리기
    # OpenCV 색상 순서는 RGB가 아니라 BGR입니다.
    # (255, 0, 0)은 빨강이 아니라 파랑입니다.
    cv2.rectangle(
        image,
        pt1=(50, 50),
        pt2=(450, 250),
        color=(255, 0, 0),
        thickness=3,
    )

    cv2.circle(
        image,
```

```

        center=(250, 150),
        radius=60,
        color=(0, 255, 0),
        thickness=-1,
    )

    cv2.putText(
        image,
        text="OpenCV Docker Test",
        org=(80, 285),
        fontFace=cv2.FONT_HERSHEY_SIMPLEX,
        fontScale=0.8,
        color=(255, 255, 255),
        thickness=2,
    )

    # 5. 원본 테스트 이미지 저장
    input_image_path = os.path.join(image_dir, "opencv_test_input.png")
    cv2.imwrite(input_image_path, image)
    print(f"테스트 이미지 저장 완료: {input_image_path}")

    # 6. OpenCV로 이미지 다시 읽기
    loaded_image = cv2.imread(input_image_path)

    if loaded_image is None:
        raise FileNotFoundError(f"이미지를 읽을 수 없습니다: {input_image_path}")

    print(f"이미지 읽기 성공: shape={loaded_image.shape}")

    # 7. 흑백 변환
    gray_image = cv2.cvtColor(loaded_image, cv2.COLOR_BGR2GRAY)

    # 8. 결과 저장
    output_image_path = os.path.join(output_dir, "opencv_test_gray.png")
    cv2.imwrite(output_image_path, gray_image)

    print(f"흑백 변환 결과 저장 완료: {output_image_path}")
    print("=== 테스트 완료 ===")

if __name__ == "__main__":
    main()

```

▼ 예시 2) Python Code

```

# app/02_image_basic_processing.py

"""
실제 이미지 파일을 OpenCV로 읽고 기본 전처리를 수행하는 예제입니다.

실습 내용:
1. 이미지 읽기
2. 이미지 크기 확인

```

3. 리사이즈

4. 중앙 영역 자르기

5. 흑백 변환

6. 결과 저장

주의:

- sample.jpg 파일은 /workspace/data/images/sample.jpg 위치에 있어야 합니다.
- Windows 기준 위치는 C:\\dev\\opencv-docker-lab\\data\\images\\sample.jpg 입니다.

```
import os
import cv2

def ensure_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)

def main():
    print("=== 실제 이미지 기본 처리 실습 시작 ===")

    input_path = "/workspace/data/images/sample.jpg"
    output_dir = "/workspace/outputs"

    ensure_dir(output_dir)

    # 1. 이미지 읽기
    image = cv2.imread(input_path)

    # 2. 이미지 읽기 실패 확인
    if image is None:
        raise FileNotFoundError(
            f"이미지를 읽을 수 없습니다: {input_path}\n"
            "확인 사항:\n"
            "1) data/images 폴더에 sample.jpg가 있는지 확인하세요.\n"
            "2) 파일명이 sample.jpg와 정확히 일치하는지 확인하세요.\n"
            "3) 확장자가 .jpeg, .png가 아닌지 확인하세요."
        )

    # 3. 이미지 크기 확인
    height, width, channels = image.shape

    print(f"원본 이미지 크기: width={width}, height={height}, channels={channels}")

    # 4. 리사이즈
    resized_width = 640
    resized_height = 480

    resized = cv2.resize(image, (resized_width, resized_height))

    resized_path = os.path.join(output_dir, "sample_resized_640x480.jpg")
    cv2.imwrite(resized_path, resized)

    print(f"리사이즈 결과 저장: {resized_path}")
```

```

# 5. 중앙 영역 자르기
# OpenCV/NumPy 배열 인덱싱은 [y1:y2, x1:x2] 순서입니다.
crop_size = 300

center_x = width // 2
center_y = height // 2

x1 = max(center_x - crop_size // 2, 0)
y1 = max(center_y - crop_size // 2, 0)
x2 = min(center_x + crop_size // 2, width)
y2 = min(center_y + crop_size // 2, height)

cropped = image[y1:y2, x1:x2]

cropped_path = os.path.join(output_dir, "sample_center_crop.jpg")
cv2.imwrite(cropped_path, cropped)

print(f"중앙 자르기 결과 저장: {cropped_path}")
print(f"자른 영역 좌표: x1={x1}, y1={y1}, x2={x2}, y2={y2}")

# 6. 흑백 변환
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

gray_path = os.path.join(output_dir, "sample_gray.jpg")
cv2.imwrite(gray_path, gray)

print(f"흑백 변환 결과 저장: {gray_path}")

print("=== 실제 이미지 기본 처리 실습 완료 ===")

if __name__ == "__main__":
    main()

```

▼ 예시 3) Python Code

```

# app/03_color_space_and_mask.py

"""
OpenCV 색상 공간 변환과 색상 기반 객체 검출 기초 실습

실습 내용:
1. BGR 테스트 이미지 생성
2. BGR 이미지를 RGB / HSV로 변환
3. 특정 색상 영역 마스크 생성
4. 마스크 결과 저장
5. 색상 검출 결과 저장

주의:
- OpenCV 기본 색상 순서는 RGB가 아니라 BGR입니다.
- HSV에서 H 범위는 0~179입니다.
- cv2.imshow()는 사용하지 않고 결과 파일을 outputs 폴더에 저장합니다.
"""

```

```

import os
import cv2
import numpy as np

def ensure_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)

def save_image(path: str, image) -> None:
    success = cv2.imwrite(path, image)
    if not success:
        raise IOError(f"이미지 저장 실패: {path}")

def main():
    print("=== 색상 공간 변환 및 색상 검출 실습 시작 ===")

    output_dir = "/workspace/outputs"
    ensure_dir(output_dir)

    # 1. 테스트 이미지 생성
    height = 400
    width = 600

    # 흰색 배경 이미지 생성
    image = np.full((height, width, 3), 255, dtype=np.uint8)

    # OpenCV 색상은 BGR 순서입니다.
    blue_bgr = (255, 0, 0)
    green_bgr = (0, 255, 0)
    red_bgr = (0, 0, 255)
    yellow_bgr = (0, 255, 255)
    black_bgr = (0, 0, 0)

    # 파란 사각형
    cv2.rectangle(
        image,
        pt1=(50, 60),
        pt2=(220, 180),
        color=blue_bgr,
        thickness=-1,
    )

    # 초록 원
    cv2.circle(
        image,
        center=(420, 120),
        radius=70,
        color=green_bgr,
        thickness=-1,
    )

    # 빨간 사각형

```



```

cv2.rectangle(
    image,
    pt1=(70, 250),
    pt2=(250, 350),
    color=red_bgr,
    thickness=-1,
)

# 노란 원
cv2.circle(
    image,
    center=(430, 300),
    radius=60,
    color=yellow_bgr,
    thickness=-1,
)

# 라벨 표시
cv2.putText(image, "BLUE", (95, 130), cv2.FONT_HERSHEY_SIMPLEX, 0.8, black_bg
r, 2)
cv2.putText(image, "GREEN", (365, 130), cv2.FONT_HERSHEY_SIMPLEX, 0.8, black_
bgr, 2)
cv2.putText(image, "RED", (125, 315), cv2.FONT_HERSHEY_SIMPLEX, 0.8, black_bg
r, 2)
cv2.putText(image, "YELLOW", (370, 310), cv2.FONT_HERSHEY_SIMPLEX, 0.8, black
_bgr, 2)

original_path = os.path.join(output_dir, "03_original_bgr.png")
save_image(original_path, image)
print(f"원본 BGR 테스트 이미지 저장: {original_path}")

# 2. BGR → RGB 변환
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 주의:
# cv2.imwrite()는 BGR 기준으로 저장하는 것이 자연스럽습니다.
# RGB 이미지를 그대로 cv2.imwrite()로 저장하면 색상이 뒤틀려 보일 수 있습니다.
# 따라서 비교용으로만 사용하고, 저장할 때는 다시 BGR로 변환합니다.
rgb_saved_as_bgr = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
rgb_path = os.path.join(output_dir, "03_rgb_converted_back_to_bgr.png")
save_image(rgb_path, rgb_saved_as_bgr)
print(f"RGB 변환 후 다시 BGR로 저장: {rgb_path}")

# 3. BGR → HSV 변환
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# HSV 이미지는 사람이 바로 보기 어렵습니다.
# 실무에서는 HSV 자체보다 마스크 결과를 주로 확인합니다.
print("BGR 이미지를 HSV 이미지로 변환 완료")

# 4. 초록색 영역 검출
# OpenCV HSV 범위:
# H: 0~179, S: 0~255, V: 0~255
lower_green = np.array([40, 80, 80])
upper_green = np.array([85, 255, 255])

```

```

green_mask = cv2.inRange(hsv_image, lower_green, upper_green)

green_mask_path = os.path.join(output_dir, "03_green_mask.png")
save_image(green_mask_path, green_mask)
print(f"초록색 마스크 저장: {green_mask_path}")

green_result = cv2.bitwise_and(image, image, mask=green_mask)

green_result_path = os.path.join(output_dir, "03_green_result.png")
save_image(green_result_path, green_result)
print(f"초록색 검출 결과 저장: {green_result_path}")

# 5. 파란색 영역 검출
lower_blue = np.array([90, 80, 80])
upper_blue = np.array([130, 255, 255])

blue_mask = cv2.inRange(hsv_image, lower_blue, upper_blue)

blue_mask_path = os.path.join(output_dir, "03_blue_mask.png")
save_image(blue_mask_path, blue_mask)
print(f"파란색 마스크 저장: {blue_mask_path}")

blue_result = cv2.bitwise_and(image, image, mask=blue_mask)

blue_result_path = os.path.join(output_dir, "03_blue_result.png")
save_image(blue_result_path, blue_result)
print(f"파란색 검출 결과 저장: {blue_result_path}")

# 6. 빨간색 영역 검출
# 빨강은 Hue 범위가 0 근처와 179 근처로 나뉩니다.
lower_red_1 = np.array([0, 80, 80])
upper_red_1 = np.array([10, 255, 255])

lower_red_2 = np.array([170, 80, 80])
upper_red_2 = np.array([179, 255, 255])

red_mask_1 = cv2.inRange(hsv_image, lower_red_1, upper_red_1)
red_mask_2 = cv2.inRange(hsv_image, lower_red_2, upper_red_2)

red_mask = cv2.bitwise_or(red_mask_1, red_mask_2)

red_mask_path = os.path.join(output_dir, "03_red_mask.png")
save_image(red_mask_path, red_mask)
print(f"빨간색 마스크 저장: {red_mask_path}")

red_result = cv2.bitwise_and(image, image, mask=red_mask)

red_result_path = os.path.join(output_dir, "03_red_result.png")
save_image(red_result_path, red_result)
print(f"빨간색 검출 결과 저장: {red_result_path}")

print("=== 색상 공간 변환 및 색상 검출 실습 완료 ===")

```

```
if __name__ == "__main__":
    main()
```

▼ 예시 4) Python Code

```
# app/04_contour_center_detection.py

"""
OpenCV 윤곽선 검출과 색상 객체 중심 좌표 계산 실습

실습 내용:
1. 색상 객체가 포함된 테스트 이미지 생성
2. BGR 이미지를 HSV로 변환
3. 특정 색상 마스크 생성
4. cv2.findContours()로 윤곽선 검출
5. 작은 노이즈 제거
6. 가장 큰 객체 선택
7. 바운딩 박스와 중심 좌표 계산
8. 결과 이미지 저장

주의:
- OpenCV 기본 색상 순서는 BGR입니다.
- HSV의 H 범위는 0~179입니다.
- cv2.imshow()는 사용하지 않고 outputs 폴더에 저장합니다.
"""

import os
import cv2
import numpy as np

def ensure_dir(path: str) -> None:    # 중복 배제?
    os.makedirs(path, exist_ok=True)

def save_image(path: str, image) -> None:
    success = cv2.imwrite(path, image)
    if not success:
        raise IOError(f"이미지 저장 실패: {path}")

def create_test_image() -> np.ndarray:    # array를 반환한다. (ndarray는 n차원 array)
    """
    실습용 테스트 이미지를 생성합니다.
    실제 카메라 영상 대신 코드에서 직접 이미지를 만들어 사용합니다.
    """

    height = 480
    width = 640

    # 흰색 배경 생성
    image = np.full((height, width, 3), 255, dtype=np.uint8) # unit8: 부호가 없는
```

정수 8비트 짜리.

```
# OpenCV는 BGR 순서입니다.
red_bgr = (0, 0, 255)
green_bgr = (0, 255, 0)
blue_bgr = (255, 0, 0)
black_bgr = (0, 0, 0)

# 검출 대상: 초록색 큰 원
cv2.circle(
    image,
    center=(420, 260),
    radius=70,
    color=green_bgr,
    thickness=-1,
)

# 검출 대상이 아닌 빨간 사각형
cv2.rectangle(
    image,
    pt1=(70, 80),
    pt2=(200, 200),
    color=red_bgr,
    thickness=-1,
)

# 검출 대상이 아닌 파란 원
cv2.circle(
    image,
    center=(180, 360),
    radius=55,
    color=blue_bgr,
    thickness=-1,
)

# 작은 초록색 노이즈들
cv2.circle(image, center=(80, 420), radius=8, color=green_bgr, thickness=-1)
cv2.circle(image, center=(580, 70), radius=6, color=green_bgr, thickness=-1)
cv2.circle(image, center=(550, 390), radius=10, color=green_bgr, thickness=-1)

# 화면 중앙 기준선 표시
cv2.line(image, (width // 2, 0), (width // 2, height), black_bgr, 1)
cv2.line(image, (0, height // 2), (width, height // 2), black_bgr, 1)

cv2.putText(
    image,
    "Target: GREEN Object",
    (20, 40),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.9,
    black_bgr,
    2,
)
```

```

return image

def detect_green_object(image: np.ndarray):
    """
    초록색 객체를 검출하고 가장 큰 객체의 정보를 반환합니다.
    """

    # 1. BGR → HSV 변환
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # 2. 초록색 HSV 범위 설정
    lower_green = np.array([40, 80, 80])
    upper_green = np.array([85, 255, 255])

    # 3. 초록색 마스크 생성
    mask = cv2.inRange(hsv, lower_green, upper_green)

    # 4. 노이즈 완화를 위한 Morphology 연산
    # 작은 점을 줄이고 객체 영역을 더 안정적으로 만듭니다.
    kernel = np.ones((5, 5), np.uint8)

    mask_clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask_clean = cv2.morphologyEx(mask_clean, cv2.MORPH_CLOSE, kernel)

    # 5. 윤곽선 찾기
    contours, _ = cv2.findContours(
        mask_clean,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE,
    )

    if not contours:
        return mask, mask_clean, None

    # 6. 작은 노이즈 제거
    min_area = 1000
    valid_contours = []

    for contour in contours:
        area = cv2.contourArea(contour)

        if area >= min_area:
            valid_contours.append(contour)

    if not valid_contours:
        return mask, mask_clean, None

    # 7. 가장 큰 객체 선택
    largest_contour = max(valid_contours, key=cv2.contourArea)

    # 8. 바운딩 박스 계산
    x, y, w, h = cv2.boundingRect(largest_contour)

    # 9. 중심 좌표 계산

```

```

center_x = x + w // 2
center_y = y + h // 2

area = cv2.contourArea(largest_contour)

object_info = {
    "contour": largest_contour,
    "area": area,
    "x": x,
    "y": y,
    "w": w,
    "h": h,
    "center_x": center_x,
    "center_y": center_y,
}

return mask, mask_clean, object_info

def draw_detection_result(image: np.ndarray, object_info):
    """
    검출 결과를 이미지 위에 시각화합니다.
    """

    result = image.copy()

    height, width = result.shape[:2]
    screen_center_x = width // 2
    screen_center_y = height // 2

    if object_info is None:
        cv2.putText(
            result,
            "No green object detected",
            (30, 80),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8,
            (0, 0, 255),
            2,
        )
        return result

    x = object_info["x"]
    y = object_info["y"]
    w = object_info["w"]
    h = object_info["h"]
    center_x = object_info["center_x"]
    center_y = object_info["center_y"]
    area = object_info["area"]

    # 바운딩 박스 그리기
    cv2.rectangle(
        result,
        (x, y),
        (x + w, y + h),

```

```

        (0, 0, 0),
        3,
    )

# 객체 중심점 그리기
cv2.circle(
    result,
    (center_x, center_y),
    8,
    (0, 0, 255),
    -1,
)

# 화면 중심에서 객체 중심까지 선 그리기
cv2.line(
    result,
    (screen_center_x, screen_center_y),
    (center_x, center_y),
    (255, 0, 255),
    2,
)

# 중심 오차 계산
error_x = center_x - screen_center_x
error_y = center_y - screen_center_y

# 정보 텍스트 출력
info_text_1 = f"center=({center_x}, {center_y}) area={area:.1f}"
info_text_2 = f"error_x={error_x}, error_y={error_y}"

cv2.putText(
    result,
    info_text_1,
    (30, 420),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 0, 0),
    2,
)

cv2.putText(
    result,
    info_text_2,
    (30, 455),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 0, 0),
    2,
)

# 로봇 제어 관점의 간단한 판단
if error_x < -50:
    command = "Object is LEFT -> turn left"
elif error_x > 50:
    command = "Object is RIGHT -> turn right"

```

```

else:
    command = "Object is CENTER -> go forward"

    cv2.putText(
        result,
        command,
        (30, 80),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 0, 0),
        2,
    )

    return result

def main():
    print("=== 윤곽선 검출 및 중심 좌표 계산 실습 시작 ===")

    output_dir = "/workspace/outputs"
    ensure_dir(output_dir)

    # 1. 테스트 이미지 생성
    image = create_test_image()

    original_path = os.path.join(output_dir, "04_original_scene.png")
    save_image(original_path, image)
    print(f"원본 테스트 이미지 저장: {original_path}")

    # 2. 초록색 객체 검출
    mask, mask_clean, object_info = detect_green_object(image)

    mask_path = os.path.join(output_dir, "04_green_mask_raw.png")
    save_image(mask_path, mask)
    print(f"초록색 원본 마스크 저장: {mask_path}")

    mask_clean_path = os.path.join(output_dir, "04_green_mask_clean.png")
    save_image(mask_clean_path, mask_clean)
    print(f"초록색 정제 마스크 저장: {mask_clean_path}")

    # 3. 검출 결과 시각화
    result = draw_detection_result(image, object_info)

    result_path = os.path.join(output_dir, "04_detection_result.png")
    save_image(result_path, result)
    print(f"검출 결과 이미지 저장: {result_path}")

    # 4. 콘솔에 객체 정보 출력
    if object_info is None:
        print("초록색 객체를 찾지 못했습니다.")
    else:
        print("초록색 객체 검출 성공")
        print(f"면적: {object_info['area']:.1f}")
        print(f"바운딩 박스: x={object_info['x']}, y={object_info['y']}, w={object_info['w']}, h={object_info['h']}")

```



```

        print(f"중심 좌표: center_x={object_info['center_x']}, center_y={object_info['center_y']}")

        height, width = image.shape[:2]
        screen_center_x = width // 2
        error_x = object_info["center_x"] - screen_center_x

        print(f"화면 중심 기준 x 오차: {error_x}")

        if error_x < -50:
            print("판단: 객체가 화면 왼쪽에 있습니다. 로봇은 왼쪽으로 회전해야 합니다.")
        elif error_x > 50:
            print("판단: 객체가 화면 오른쪽에 있습니다. 로봇은 오른쪽으로 회전해야 합니다.")
        else:
            print("판단: 객체가 화면 중앙에 있습니다. 로봇은 전진 또는 집기 동작을 수행할 수 있습니다.")

        print("=== 윤곽선 검출 및 중심 좌표 계산 실습 완료 ===")

if __name__ == "__main__":
    main()

```

▼ 예시 5) Python Code

```

# app/05_canny_edge_detection.py

"""
OpenCV 이미지 필터링과 Canny Edge Detection 실습

실습 내용:
1. 테스트 이미지 생성
2. 흑백 변환
3. Gaussian Blur 적용
4. Canny Edge Detection 적용
5. threshold 값 변화 비교
6. 결과 이미지 저장

주의:
- cv2.imshow()는 사용하지 않고 outputs 폴더에 저장합니다.
- Docker 컨테이너 내부 기준 출력 경로는 /workspace/outputs 입니다.
"""

import os
import cv2
import numpy as np

def ensure_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)

```

```

def save_image(path: str, image) -> None:
    success = cv2.imwrite(path, image)
    if not success:
        raise IOError(f"이미지 저장 실패: {path}")

def create_edge_test_image() -> np.ndarray:
    """
    에지 검출 실습용 테스트 이미지를 생성합니다.
    실제 카메라 이미지 대신 도형과 선이 포함된 이미지를 만듭니다.
    """

    height = 480
    width = 640

    # 흰색 배경
    image = np.full((height, width, 3), 255, dtype=np.uint8)

    black = (0, 0, 0)
    red = (0, 0, 255)
    blue = (255, 0, 0)
    green = (0, 255, 0)
    gray = (180, 180, 180)

    # 큰 사각형
    cv2.rectangle(image, (60, 80), (240, 240), black, 3)

    # 채워진 원
    cv2.circle(image, (430, 160), 80, blue, -1)

    # 삼각형
    points = np.array([[320, 330], [460, 430], [200, 430]], np.int32)
    points = points.reshape((-1, 1, 2))
    cv2.polylines(image, [points], isClosed=True, color=red, thickness=4)

    # 로봇 주행 라인처럼 보이는 두 선
    cv2.line(image, (90, 460), (280, 280), green, 5)
    cv2.line(image, (550, 460), (360, 280), green, 5)

    # 약한 회색 선
    cv2.line(image, (20, 40), (620, 40), gray, 2)

    # 작은 노이즈 점 생성
    rng = np.random.default_rng(seed=42)

    for _ in range(200):
        x = int(rng.integers(0, width))
        y = int(rng.integers(0, height))
        color_value = int(rng.integers(0, 255))
        image[y, x] = (color_value, color_value, color_value)

    cv2.putText(
        image,
        "Canny Edge Test",
        (190, 40),

```

```

        cv2.FONT_HERSHEY_SIMPLEX,
        0.9,
        black,
        2,
    )

    return image

def main():
    print("=== Canny Edge Detection 실습 시작 ===")

    output_dir = "/workspace/outputs"
    ensure_dir(output_dir)

    # 1. 테스트 이미지 생성
    image = create_edge_test_image()

    original_path = os.path.join(output_dir, "05_original_edge_test.png")
    save_image(original_path, image)
    print(f"원본 이미지 저장: {original_path}")

    # 2. 흑백 변환
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    gray_path = os.path.join(output_dir, "05_gray.png")
    save_image(gray_path, gray)
    print(f"흑백 이미지 저장: {gray_path}")

    # 3. Gaussian Blur 적용
    # 커널 크기는 홀수여야 합니다. 예: 3, 5, 7
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    blurred_path = os.path.join(output_dir, "05_blurred.png")
    save_image(blurred_path, blurred)
    print(f"Gaussian Blur 이미지 저장: {blurred_path}")

    # 4. Canny Edge Detection 적용
    edges_50_150 = cv2.Canny(blurred, 50, 150)

    edges_50_150_path = os.path.join(output_dir, "05_canny_50_150.png")
    save_image(edges_50_150_path, edges_50_150)
    print(f"Canny 결과 저장 threshold 50/150: {edges_50_150_path}")

    # 5. threshold 값 변화 비교
    edges_30_100 = cv2.Canny(blurred, 30, 100)
    edges_100_200 = cv2.Canny(blurred, 100, 200)
    edges_150_250 = cv2.Canny(blurred, 150, 250)

    save_image(os.path.join(output_dir, "05_canny_30_100_more_sensitive.png"), edges_30_100)
    save_image(os.path.join(output_dir, "05_canny_100_200_standard.png"), edges_100_200)
    save_image(os.path.join(output_dir, "05_canny_150_250_less_sensitive.png"), edges_150_250)

```

```

print("threshold 비교 결과 저장 완료")

# 6. Blur 없이 Canny 적용해서 비교
edges_without_blur = cv2.Canny(gray, 50, 150)

edges_without_blur_path = os.path.join(output_dir, "05_canny_without_blur.png")
save_image(edges_without_blur_path, edges_without_blur)
print(f"Blur 없이 Canny 적용 결과 저장: {edges_without_blur_path}")

# 7. 컬러 원본 위에 에지 표시
# edges는 흑백 이미지가므로 컬러 마스크처럼 활용합니다.
edge_overlay = image.copy()
edge_overlay[edges_50_150 > 0] = (0, 0, 255)

overlay_path = os.path.join(output_dir, "05_edge_overlay_red.png")
save_image(overlay_path, edge_overlay)
print(f"에지 오버레이 이미지 저장: {overlay_path}")

print("=== Canny Edge Detection 실습 완료 ===")

if __name__ == "__main__":
    main()

```

AI 프롬프트 예시

나는 antigravity 사용.

Windows 10 PRO + Docker Compose + Python + OpenCV 환경에서
Gaussian Blur와 Canny Edge Detection을 실습하는 초보자용 Python 코드를 작성해 주세요.

조건:

- 테스트 이미지는 코드에서 자동 생성
- cv2.imshow()는 사용하지 않음
- 결과는 /workspace/outputs에 저장
- threshold(기준값) 30/100, 50/150, 100/200, 150/250을 비교
- Blur 적용 전후 Canny 결과를 비교
- ROS2 로봇 비전에서 어떤 의미인지 주석으로 설명

OpenCV 내용이 너무 많아져서 OpenCV 내용만 정리하는 파일을 하나 더 만들겠음.